

ALGORITMO Codificación Huffman

INICIO

ENTRADA: Texto

SALIDA: Frecuencias de caracteres, Códigos Huffman, Texto codificado, Árbol

1. Leer texto desde consola

2. Crear un diccionario (mapa) de frecuencias

3. PARA cada caracter c en el texto HACER

 aumentar contador de frecuencias[c]

FIN PARA

4. Construir el árbol de Huffman:

- Crear una PriorityQueue de Nodos (ordenada por frecuencia)

- Insertar cada caracter con su frecuencia como nodo hoja

- MIENTRAS haya más de un nodo en la cola HACER

 sacar los dos nodos con menor frecuencia

 crear nodo padre con frecuencia = suma de ambos

 asignar hijos izquierdo y derecho

 insertar nodo padre en la cola

FIN MIENTRAS

- El nodo restante en la cola es la raíz del árbol

5. Generar códigos de Huffman:

- Iniciar recorrido recursivo desde la raíz

- Si el nodo es hoja:

 asignar código al caracter (concatenación de 0s y 1s según camino)

EN CASO CONTRARIO:

 bajar a hijo izquierdo con código + "0"

bajar a hijo derecho con código + "1"

6. Codificar el texto original:

- Para cada caracter c en el texto
concatenar su código de Huffman

7. Imprimir resultados:

- Tabla de frecuencias
- Códigos de Huffman
- Texto codificado
- Árbol de Huffman con indentación (usar método imprimirArbol)

FIN ALGORITMO

Clase Nodo:

// Clase Nodo del árbol de Huffman

```
class Nodo implements Comparable<Nodo> {
```

```
    char caracter;    // caracter (si es hoja)
```

```
    int frecuencia;   // frecuencia del caracter o suma de frecuencias
```

```
    Nodo izquierda;   // hijo izquierdo
```

```
    Nodo derecha;     // hijo derecho
```

// Constructor para hoja

```
Nodo(char caracter, int frecuencia) {
```

```
    this.caracter = caracter;
```

```
    this.frecuencia = frecuencia;
```

```
}
```

// Constructor para nodo interno

```
Nodo(int frecuencia, Nodo izquierda, Nodo derecha) {
```

```

        this.caracter = '\0'; // nodo interno no tiene caracter

        this.frecuencia = frecuencia;

        this.izquierda = izquierda;

        this.derecha = derecha;
    }

    // Para que PriorityQueue ordene por frecuencia (menor primero)
    public int compareTo(Nodo o) {
        return this.frecuencia - o.frecuencia;
    }
}

```

Método para imprimir el árbol

// Método para imprimir árbol con indentación

```

public static void imprimirArbol(Nodo raiz, String prefijo) {
    if (raiz == null) return;

    if (raiz.caracter != '\0') { // nodo hoja
        System.out.println(prefijo + "└─ " + raiz.caracter + " (" + raiz.frecuencia + ")");
    } else { // nodo interno
        System.out.println(prefijo + "└─ * (" + raiz.frecuencia + ")");
    }

    imprimirArbol(raiz.izquierda, prefijo + "  ");
    imprimirArbol(raiz.derecha, prefijo + "  ");
}

```

Ejemplo de ejecución:

Ingresa el texto: bbbbaaccde

Frecuencias: {a=2, b=4, c=2, d=1, e=1}

Códigos de Huffman: {a=110, b=0, c=10, d=1110, e=1111}

Texto codificado: 0000110110101011101111

Árbol de Huffman:

